

FULL STACK DEVELOPMENT

FASE 5 | AULA 08 -

CLEAN ARCHITECTURE - IMPLEMENTANDO A ARQUITETURA

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS.....	11

EXEMPLO

O QUE VEM POR AÍ?

É hora de revisar. Vamos lembrar o que dissemos desde a primeira aula da disciplina, recapitular os principais pontos e, ao final, entender qual o papel de uma boa arquitetura no projeto de software, além do motivo de a Arquitetura Limpa definir um conjunto de ótimos conceitos para nos ajudar na implementação.

EXEMPLO

HANDS ON

Vamos acompanhar uma implementação de código para investigar como as regras da Arquitetura Limpa se aplicam na codificação.

Para essa aula, temos um link para você poder treinar o conteúdo ensinado.

Acesse:

[Base de dados aula 08.](#)

EXEMPLO

SAIBA MAIS

Construir um software é difícil, construir um com qualidade é mais difícil ainda e construir um de qualidade, que pode ser alterado e entendido facilmente, tende a ser uma tarefa exaustiva e muitas vezes demorada. Isso é esperado e é importante entender que uma parte do profissionalismo de qualquer pessoa na indústria de software vem da capacidade de produzir códigos, ao passo que a outra vem da perícia em criar códigos de alta qualidade no tempo adequado.

A Arquitetura Limpa define uma série de ideias importantes para apoiar a construção de software de alta qualidade, focando na visão arquitetural do produto de software em construção. Mais do que indicar qual componente deve ir em cada lugar da arquitetura, há uma série de ideias que servem como uma heurística para as decisões que devem ser tomadas durante a construção do software.

SOLID

Falamos inicialmente sobre o SOLID, acrônimo que define cinco regras de construção de componentes de software que, como vimos em toda a disciplina, são seguidas também na definição de como estruturar o projeto.

- O “**S**” corresponde ao Princípio de Responsabilidade Única e define que nenhum componente deve fazer mais de uma coisa, ou seja: cada componente deve ter uma única responsabilidade e, mais do que isso, atender a apenas um tipo de requisição. Isso permite criar componentes que são altamente especializados no que fazem e como se conectam a outros componentes.
- O “**O**” corresponde ao Princípio Aberto-Fechado, que define que os componentes devem ser fechados para alteração e abertos para extensão. Isso porque é preferível implementar uma nova funcionalidade em um novo componente, não inserindo mais complexidade em um componente já existente. E para que isso seja possível, o componente original deve permitir ser utilizado por outros componentes, seja via herança ou composição. Bons componentes que seguem esse princípio permitem a criação de novos com base na composição de componentes já existentes.

- O “**L**” corresponde ao Princípio de Substituição de Liskov, em que devemos garantir que uma classe derivada possa ser substituída por sua classe base sem que o software pare de funcionar. Apesar de um pouco estranha no começo, essa regra é importante para garantir que uma classe não dependa, de modo indireto, de um comportamento de uma classe derivada. Assim, podemos injetar dependências com mais segurança.
- O “**I**”, de Princípio de Segregação de Interface, define que uma classe não deve depender de um comportamento que não usa. Ou seja: qualquer dependência dessa classe deve ser adaptada à interface e não a um objeto específico. Por isso, sempre que esperamos algum objeto, devemos nos referir ao comportamento dele (definido na interface) e não à forma. O uso de interfaces permite que possamos passar, como dependências, objetos que se adaptam a um conjunto de métodos (comportamento) utilizados pela classe, não importando qual a implementação concreta. Isso é especialmente útil na injeção de dependências e na comunicação entre componentes, que é baseada em **protocolos**.
- E por último, o “**D**”, que corresponde ao Princípio de Inversão de Dependência. Ele define que um objeto de mais alto nível não pode depender diretamente de um objeto de mais baixo nível ou, ainda, que um objeto que implementa detalhes de uma implementação deve depender apenas de abstrações, como interfaces. Isso permite que um componente x não dependa forte e diretamente de outro componente y; isso porque, caso esse componente y mude, o componente x pode parar de funcionar; o ideal é que o componente x dependa apenas dos comportamentos que ele usa do componente y, ou seja: de uma interface, e que ele idealmente receba este componente como dependência no momento do uso.

Arquitetura Limpa, camadas e componentes

Com base na figura 1 – “Diagrama de referência da Arquitetura Limpa”, analisamos as camadas e tipos de componentes existentes em cada uma dessas camadas.

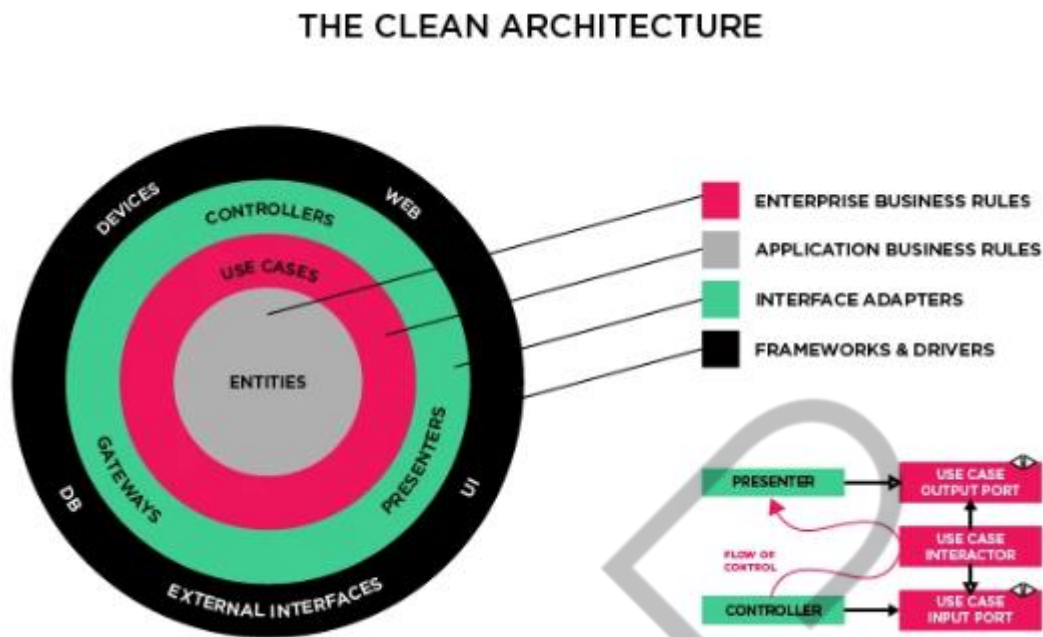


Figura 1 - Diagrama de Referência - Arquitetura Limpa
Fonte: Martin (2019), adaptado por FIAP (2023)

- **Entidades:** a camada mais interna no diagrama central é a que define os elementos únicos da regra de negócio, as Entidades. Cada uma delas representa a abstração de um conceito definido nos casos de uso de uma regra de negócio. No caso de um software para uma escola, as entidades podem ser Estudante, Disciplina, Docente, Avaliação. Cada Entidade é responsável por encapsular os dados relativos a esse conceito, fornecer mecanismos que garantam seu estado interno consistente (a validação dos dados em todo o ciclo de vida dessa entidade) e validar o comportamento da colaboração com outras entidades.
- **Casos de Uso:** a camada exterior à camada de Entidades tem os componentes de software que encapsulam cada um dos casos de uso encontrados na regra de negócio. Aqui, temos a implementação das funcionalidades que o software entrega, uma em cada componente, para que sejam utilizadas de forma independente ou interdependente. Cada caso de uso aqui definido utiliza os dados que recebe das camadas exteriores, instancia as entidades necessárias para executar seu trabalho e executa as tarefas definidas, respondendo para o componente que o requisitou. Alguns exemplos de caso de uso podem ser Aplicar Avaliação e Matricular Estudante em Disciplina.

- **Adaptadores de Interface:** contando de dentro para fora, a terceira camada é responsável por disponibilizar, para as camadas exteriores, o acesso aos casos de uso. Nela, o trabalho principal é fazer com que as requisições sejam atendidas adequadamente e para isso ela conta com três tipos de componentes: os **Controllers**, que servem para receber as requisições, preparar os dados recebidos e executar os casos de uso necessários; os **Gateways**, que servem para criar abstrações de recursos de acesso a dados para que os casos de uso possam utilizar; e os **Adapters**, usados pelos Controllers para adaptar os dados para o formato necessário no retorno de dados de um controller para seu requisitante.
- **Frameworks e Drivers:** a camada mais externa. É a que implementa o acesso aos recursos externos ao nosso software; é nela que estão a implementação da camada de APIs abertas a usuários, os componentes usados para acesso a serviços de repositórios de dados, o acesso a serviços externos utilizados nas regras de negócio e tudo mais que o software precisa para funcionar.

Dentro de todas essas camadas, temos os diversos componentes que implementam as responsabilidades, ou seja, que são o código em si. Seguimos algumas regras na criação e na definição da interação entre os componentes:

1. Os componentes internos não devem depender diretamente nem acessar os componentes externos. Caso precisem, o componente necessário será informado via injeção de dependência e os componentes internos devem sempre definir uma interface para “receber” este componente externo.
2. Os componentes da mesma camada podem interagir diretamente e depender entre si para completar uma requisição recebida. Um caso de uso pode chamar outro, caso necessário; ou um Controller vai usar um Adapter para preparar os dados de resposta de uma requisição.

Para o software que estamos construindo de acordo com os preceitos da Arquitetura Limpa, os recursos externos são vistos como serviços externos e não como dependências; com isso, modelamos a solução para que a operação destes recursos e suas características sejam vistos como detalhes, ou seja: que a arquitetura se importa em como operar estes recursos, não como eles funcionam ou algo assim.

Dessa forma, conseguimos fazer com que nossa Arquitetura Limpa entregue um software escalável de alta qualidade de funcionamento e manutenção.

EXEMPLO

O QUE VOCÊ VIU NESTA AULA?

Nessa aula, relembramos os principais conceitos que a Arquitetura Limpa determina, indo desde as suas camadas até regras de utilização, e principalmente o framework SOLID, que serve de base para toda essa arquitetura.

O que achou desse conteúdo? Conte-nos no Discord! Estamos à disposição para tirar dúvidas, enviar avisos, fazer networking e muito mais.



REFERÊNCIAS

EVANS, E. **Domain-Driven Design**: Tackling Complexity in the Heart of Software. [s.l.]: Addison-Wesley Professional, 2003.

MARTIN, R. **Arquitetura Limpa**. Rio de Janeiro: Alta Books Editora, 2019.

MARTIN, Ro. **Clean Code**: A Handbook of Agile Software Craftsmanship. [s.l.]: Pearson, 2008.

EVANS

PALAVRAS-CHAVE

Arquitetura Limpa. Camadas. SOLID.

EXEMPLO

POS TECH